

FASE 1 - Machine Learning - Aula 3 - Redução de Dimensionalidade

<https://on.fiap.com.br/mod/conteudoshtml/view.php?id=520859&sesskey=Q9T2x0SrAc>

Assistindo os vídeos, 01,02 e 03

PCA - Principal Component Analysis (Análise do Componente Principal)

- Técnica da estatística para reduzir a dimensionalidade dos dados
- Importancia de padronizar os tipos de dados (padronização e normalização)
 - Ajustar a escala dos dados
 - O PCA é sensível a escala de dados
 - É preciso retirar os outliers
 - kg, grama, cm, metro
- A padronização de dados com o Python
 - `from sklearn.preprocessing import StandardScaler`
- O PCA vai reduzir a dimensionalidade 4 dimensionalidades para 2 dimensionalidades para daí poder seguir com outros processamentos com outros algoritmos em ML.
- PC1, PC2, PCx, cria uma nova representação dos dados com base na variabilidade.
- Importante fazer análise de correlação e plotar uma matrix usando seaborn

> Ao fazer correção de dados utilizar a biblioteca do Pandas para isso

```
df.corr()
```

> Aplicar o gráfico de correlação e visualização em forma de calor para ajudar na identificação das maiores correlações e para ajudar a na documentação com prints e evidências. Para isso usar a biblioteca "seaborn"

Exemplo

```
import seaborn as sns
plt.figure(figsize=(10,8))
sns.heatmap(colleation_matrix, cmp='coolwarm', annot=False, fmt=".2f", linewidths=.5)
plt.title("Matriz de Correlação")
plt.show()
```

Foi usado o Shapiro para detectar para cada PC (principal componente) se segue uma distribuição normal “gaussiana” para ver o quanto os dados estão preparados e aderentes ou compatível para os melhores resultados para aplicação de métodos estatísticos

Quanto mais Gaussiano melhor!

07-10-2025

Baixar o Github e testar todos os códigos

https://github.com/FIAP/CURSO_IA_ML

- ok - executado o exemplo 2
- ok - executado o exemplo 3
- ok - executado o exemplo 4

Normalização e Padronização são técnicas essenciais de **escalonamento de dados** (ou *feature scaling*) no Machine Learning, usadas para garantir que as variáveis de diferentes escalas contribuam de forma equilibrada para o treinamento do modelo.

A diferença principal reside no **intervalo de valores** e na **sensibilidade a outliers**:

1. Normalização (Min-Max Scaling)

A Normalização, geralmente referida como *Min-Max Scaling*, transforma os dados para que fiquem dentro de um **intervalo fixo**, tipicamente entre **0 e 1**.

Característica	Descrição
Fórmula	

Intervalo Final	 (ou  se houver valores negativos nos dados originais).
Sensibilidade a Outliers	Muito Sensível. Como a fórmula usa os valores mínimo e máximo, um <i>outlier</i> extremo pode "achatar" todos os outros dados para um intervalo muito estreito, perdendo-se a informação útil.
Quando Usar	É ideal quando a distribuição dos dados não é Gaussiana (normal) e quando você sabe que os limites do intervalo (0 e 1) são aceitáveis para o algoritmo (como em Redes Neurais com funções de ativação Sigmoid/Tanh).
Implementação (Scikit-learn)	MinMaxScaler

2. Padronização (Z-Score Standardization)

A Padronização, também conhecida como *Z-Score Standardization*, transforma os dados

para que tenham uma **média** () de 0 e um **desvio padrão** () de 1.

Característica	Descrição
Fórmula	

Intervalo Final	Não há um intervalo fixo; os valores transformados podem ser qualquer número .
Sensibilidade a Outliers	Menos Sensível que a Normalização. Como usa a média e o desvio padrão (que são afetados por <i>outliers</i> , mas menos drasticamente que o valor máximo), ela preserva melhor as informações sobre <i>outliers</i> .
Quando Usar	É mais adequada quando os dados seguem uma distribuição Gaussiana (normal) ou quando o algoritmo assume que os dados estão centrados em 0 e têm variância unitária (como Regressão Linear, Regressão Logística e PCA).
Implementação (Scikit-learn)	<code>StandardScaler</code>

Resumo e Guia Rápido

Técnica	Objetivo	Onde Usar	Efeito em Outliers
Normalização	Escalar para um intervalo fixo (geralmente ).	Distribuições não-normais , algoritmos que exigem limites (e.g., Redes Neurais).	Muito Sensível. Outliers podem distorcer o intervalo.

Padronização	Transformar para  e  .	Distribuições aproximadamente normais , modelos que usam distância ou assumem normalidade (e.g., PCA, Regressão, SVM).	Menos Sensível. Mantém a informação útil dos <i>outliers</i> .
---------------------	--	---	--

Qual Escolher?

Na prática de Machine Learning, a regra geral é:

1. **Padronização** é o ponto de partida mais seguro (default), especialmente se você não sabe a distribuição dos dados ou se eles possuem *outliers*.
2. **Normalização** é geralmente reservada para Redes Neurais (onde o intervalo de 0 a 1 é preferido para a convergência) ou quando você precisa garantir que os dados estejam estritamente dentro de um intervalo.

O melhor método é sempre aquele que resulta na melhor performance para o seu modelo, o que significa que, em muitos casos, você deve **testar ambos**.

O que é o modelo de Churn

O **Modelo de Churn** (ou **Modelo de Predição de Churn**) é uma ferramenta de análise preditiva, frequentemente construída com técnicas de Machine Learning, que tem como objetivo principal **identificar e prever quais clientes têm maior probabilidade de abandonar** o produto ou serviço de uma empresa em um futuro próximo.

Em essência, ele transforma dados históricos do cliente em uma **probabilidade de risco de perda**.

O Conceito Por Trás do Modelo

1. **Churn (Abandono):** É a taxa ou o número de clientes que param de fazer negócios com uma empresa em um período específico. Reduzir o *churn* é crucial, pois adquirir novos clientes costuma ser mais caro do que reter os existentes.
2. **Modelo Preditivo:** O modelo de *churn* analisa o **comportamento passado** de clientes que cancelaram (os "churners") e clientes que permaneceram (os "non-churners"). Ele identifica padrões nos dados, como:
 - Frequência de uso do produto.

- Histórico de reclamações no suporte.
 - Mudanças no plano de assinatura.
 - Informações demográficas e de contrato.
3. **Saída do Modelo:** O resultado mais comum é uma **pontuação de risco** (por exemplo, 0 a 100) ou uma **probabilidade** (por exemplo, 85% de chance de *churn*) para cada cliente ativo.

Como o Machine Learning é Aplicado

A construção de um Modelo de Churn é um problema clássico de **Classificação** no Machine Learning: o objetivo é classificar cada cliente em uma de duas categorias: "**vai fazer churn**" (Sim/1) ou "**não vai fazer churn**" (Não/0).

Algoritmos comuns utilizados incluem:

- **Regressão Logística:** Um modelo estatístico que estima a probabilidade de um evento. É um dos modelos mais interpretáveis.
- **Random Forest (Floresta Aleatória):** Um algoritmo de aprendizado de conjunto (ensemble) que oferece alta precisão e pode indicar quais fatores (recursos) são mais importantes na decisão de *churn*.
- **Gradient Boosting (e suas variações, como XGBoost/LightGBM):** Frequentemente fornecem a maior precisão em muitos conjuntos de dados, sendo ideais para prever o risco com alta acurácia.

Benefícios para as Empresas

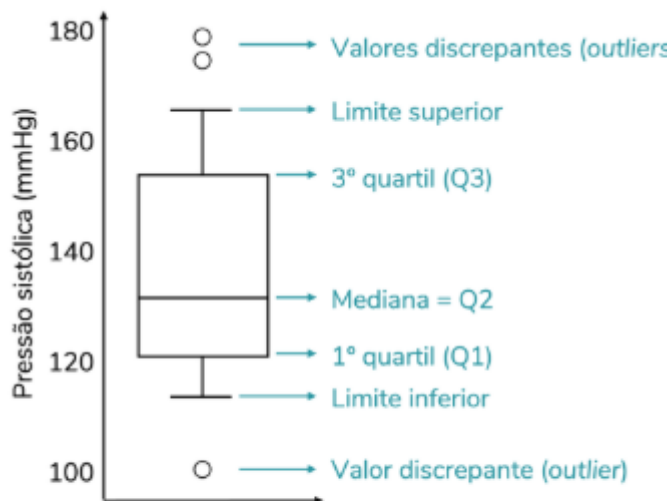
A principal vantagem de ter um Modelo de Churn é a **capacidade de antecipação**, o que permite à empresa tomar medidas de retenção de forma proativa.

- **Intervenção Proativa:** Em vez de reagir ao cancelamento, a empresa identifica clientes com alto risco de *churn* e envia ofertas de retenção, suporte personalizado ou recursos educacionais antes que o cliente decida sair.
- **Alocação de Recursos:** O modelo permite que a equipe de *Customer Success* (Sucesso do Cliente) concentre seus esforços e recursos limitados nos clientes que realmente precisam de atenção (aqueles com alto risco).
- **Entendimento do Cliente:** Ao analisar o modelo, a empresa descobre quais **fatores** (reclamações, inatividade, preço) são os principais impulsionadores do *churn*, permitindo melhorias estratégicas no produto ou serviço.

Portanto, o Modelo de Churn é uma ferramenta estratégica que transforma dados brutos em inteligência acionável para proteger a base de receita da empresa.

Anatomia de um gráfico BoxPlot

<https://fernandaferes.com.br/blog/interpretacao-boxplot/>



Elementos que compõem um boxplot.

Label Encoding

transformações nos dados, os dados em formato de string (as categorias) também passam por um tipo de transformação. Vamos aplicar nos dados em formato de texto o **LabelEncoder**. O LabelEncoder transforma rótulos de classes em números inteiros.

Mas por que é importante fazer esse tipo de transformação nas categorias? 🤔

Para os algoritmos de machine learning funcionarem, é necessário **transformar a informação em um formato numérico para que o computador possa compreender** o que estamos querendo apresentar.

Label Encoding **não** é uma técnica de padronização nem de normalização.

O que é Label Encoding?

Label Encoding é uma técnica de **codificação de variáveis categóricas**.

Seu objetivo é transformar rótulos de texto (como "vermelho", "verde", "azul") em valores numéricos inteiros (como 1, 2, 3) para que possam ser processados por algoritmos de Machine Learning.

Exemplo:

Categoria Original	Label Encoding
"Solteiro"	0
"Casado"	1
"Divorciado"	2

Diferença Fundamental

A confusão surge porque as três técnicas envolvem a manipulação de números, mas seus **propósitos** são totalmente diferentes:

1. **Label Encoding (Codificação):**
 - **Propósito:** Transformar **texto em número**.
 - **Onde Usar:** Na etapa de pré-processamento, para converter variáveis categóricas em um formato que o modelo possa ler.
2. **Normalização/Padronização (Escala de Recursos):**
 - **Propósito:** Mudar a **escala** de variáveis **numéricas** existentes.
 - **Onde Usar:** Após a codificação (se necessário), para garantir que as variáveis numéricas estejam em uma faixa ou distribuição semelhante, evitando que variáveis com valores maiores dominem o modelo.

Em resumo:

- **Label Encoding** lida com a **natureza do dado** (transformando *strings* em *inteiros*).
- **Normalização/Padronização** lida com a **magnitude do dado** (ajustando a *escala* dos *inteiros* ou *floats*).